

Curso Inicial Front-End

Clase 04: HTML

Semántico +

Formularios +

Multimedia

Módulo 1: Arquitectura y Maquetación Moderna (HTML y CSS)



Presentación

Ya sabemos poner etiquetas, ahora aprenderemos a darles **Sentido**. En esta clase exploraremos la **Semántica HTML5**, vital para que Google entienda tu web (SEO) y para que personas con discapacidad puedan navegarla. Además, dominaremos el universo de los **Formularios**, la herramienta principal para recolectar datos de tus usuarios.



Objetivos

- Implementar etiquetas semánticas (`<header>`, `<main>`, `<aside>`, `<article>`) de forma correcta.
- Dominar el catálogo exhaustivo de tipos de `input` (email, date, color, etc.) y sus validaciones.
- Comprender la accesibilidad web básica mediante atributos ARIA y semántica.



Bloques Temáticos

- Tema 1: Organización de la información: header, main, footer, section, article.
 - HTML Semántico: El Poder del Significado
- Tema 2: Entradas de datos (Inputs) y validaciones nativas.
 - Formularios: Tu buzón de recolección de datos
- Tema 3: Implementación de audio, video y figuras.
 - Multimedia, Elementos Embebidos y Control de Visibilidad

En la primera clase aprendimos a poner bloques básicos en nuestra web usando `<div>` y `<p>`. Sin embargo, si construyes un edificio solo con ladrillos genéricos sin poner carteles de "Salida", "Baño" o "Escaleras", los visitantes se perderán. En la web pasa lo mismo.

Hoy aprenderemos a construir sitios web modernos que no solo "se ven bien", sino que son **inteligentes**, accesibles para todos los usuarios y fáciles de entender para los robots de búsqueda de Google.

Tema 1: Organización de la información: header, main, footer, section, article.

1.1. HTML Semántico: El Poder del Significado

Para construir interfaces web profesionales y modernas, no basta con agrupar elementos de manera visual en la pantalla. Es indispensable comprender cómo estructurar la información para que sea legible tanto para los usuarios como para los motores de búsqueda y las tecnologías asistivas. El verdadero desarrollo web profesional comienza cuando dejamos atrás las viejas prácticas de maquetación y adoptamos un enfoque estrictamente semántico.

En el principio de la web, los programadores usaban la etiqueta genérica `<div>` (que es simplemente una "caja vacía") para envolver absolutamente todo. Un botón era un `div`, el menú era un `div`, el pie de página era un `div`. Esta práctica (conocida peyorativamente como "div-soup" o sopa de divs) es un grave error hoy en día. Una web profesional debe ser estrictamente **Semántica**.

- **Semántica:** Significa utilizar la etiqueta correcta que describa exactamente el *significado* del contenido que lleva dentro, y no solo preocuparnos por cómo se ve visualmente.
- **¿Por qué es vital usar semántica?**
 1. **SEO (Posicionamiento en Google):** Los robots exploradores de Google (crawlers) leen el código de tu sitio, no lo "miran" con ojos. Si pones todo en `div`, para el robot todo tiene la misma importancia (ninguna). Si usas una etiqueta `<main>`, el robot sabe de inmediato que

allí está el "corazón" o contenido principal de tu web, premiándote en los resultados de búsqueda.

2. **Accesibilidad (A11y):** Las personas con discapacidades visuales utilizan programas llamados "Lectores de Pantalla" que les "leen" la web en voz alta. Gracias a las etiquetas semánticas, el lector puede guiarlos: *"Estás entrando en la barra de navegación principal"*, en lugar de simplemente decir *"caja"*.

 **Recurso Oficial:** [MDN Web Docs - Elementos Semánticos](#)

Para entender su impacto en el desarrollo de software, analicemos sus ventajas y limitaciones:

✓ Ventajas (Pros)	✗ Desventajas (Contras)
Accesibilidad perfecta: Los lectores de pantalla para personas ciegas entienden y navegan la estructura al instante.	Curva de aprendizaje inicial: Exige que el programador estudie y memorice más etiquetas específicas en lugar de usar un <code>div</code> perezoso para todo.
SEO Mejorado exponencialmente: Google posicionará mucho mejor tu página frente a competidores con código pobre.	Ninguna desventaja técnica: Usar semántica no tiene costo de rendimiento ni hace la web más lenta. Es pura disciplina.

Las Etiquetas que dan Arquitectura (Layout):

En lugar de `divs`, usa estas etiquetas para estructurar el esqueleto de tu página web:

- **<header>**: El encabezado del sitio (suele contener el logo y el título principal de la página).
- **<nav>**: La barra de navegación (el menú donde están los links de "Inicio", "Contacto", etc).
- **<main>**: El contenido principal y único de la página.
- **<aside>**: Contenido relacionado pero secundario. Ideal para barras laterales, menús de navegación lateral, publicidad o widgets de "entradas recientes".
- **<footer>**: El pie de página con info de contacto o enlaces legales.
- **<section>**: Una gran sección temática de la web (ej: "Nuestros Servicios" o "Testimonios").
- **<article>**: Un contenido independiente que tendría sentido si lo cortas y lo pegas en otro lado (ej: un post de un blog o una noticia).

A veces las etiquetas de **HTML** no alcanzan para explicarle la web a un no vidente, o nos vemos forzados a usar componentes complejos contruidos con **JS**. Para eso existen los atributos **ARIA (Accessible Rich Internet Applications)**. Son atributos mágicos que hablan directamente con el lector de pantalla:

- **role="button"**: Le avisa al lector que este elemento actúa como un botón interactivo (incluso si por error visual lo programamos sobre un **div**).
- **aria-label="Cerrar ventana emergente"**: Le otorga un nombre descriptivo a un botón que visualmente solo tiene un icono de una "X". ¡El usuario invidente ahora sabe qué hace ese botón misterioso!
- **tabindex**: Permite que un elemento sea "enfocable" cuando el usuario navega usando la tecla **TAB** del teclado (muy común en accesibilidad motriz). Si pones **tabindex="0"** entra en el orden natural, si pones **tabindex="-1"** lo vuelves ignorado por el teclado.

Escribir un código semántico y accesible transforma por completo la calidad de un proyecto web. Al reemplazar las estructuras vacías por etiquetas con significado real y potenciar la accesibilidad con atributos ARIA, garantizamos que nuestro sitio esté listo para los estándares profesionales del mercado. Una vez consolidada la arquitectura semántica del texto, el siguiente paso es enriquecer la experiencia del usuario incorporando elementos visuales e interactivos modernos.

Tema 2: Entradas de datos (Inputs) y validaciones nativas.

2.1. Formularios: Tu buzón de recolección de datos

Hasta ahora, la comunicación de nuestra web era unidireccional: nosotros le mostrábamos cosas al usuario. Con los formularios, **el usuario puede**

Los Controles de Usuario: El universo `<input>`

El elemento `<input>` es el más versátil de **HTML**. Su comportamiento cambia radicalmente según el atributo `type`. Aquí tienes los más importantes que debes conocer:

Tipo (type)	Uso Principal	Características
<code>text</code>	Nombres, búsquedas.	El valor por defecto. Texto simple.
<code>password</code>	Contraseñas.	Oculto los caracteres con puntos o asteriscos.
<code>email</code>	Correos electrónicos.	Valida automáticamente que tenga un formato <code>@</code> y <code>.com</code> .
<code>number</code>	Edades, cantidades.	Muestra flechitas para subir/bajar y solo permite números.

Tipo (type)	Uso Principal	Características
tel	Teléfonos.	Abre el teclado numérico en celulares.
date	Fechas.	Abre un calendario desplegable nativo del navegador.
color	Selector de color.	Abre una paleta de colores para que el usuario elija.
checkbox	Opciones múltiples.	Permite seleccionar varias opciones al mismo tiempo.
radio	Opción única.	Obliga a elegir solo una opción de un grupo (ej: Género).
range	Deslizador.	Útil para volumen, precios o filtros de rango.
file	Subida de archivos.	Abre el explorador de archivos del sistema.
submit	Botón de envío.	El botón que gatilla la acción de enviar el formulario.

 IMPORTANT

Atributo Name: Nunca olvides poner el atributo `name` a tus inputs. Sin él, el servidor no sabrá qué dato es cuál cuando reciba el formulario.

La conexión vital de usabilidad: Label e Input

El `<input>` es el cuadrado blanco donde la persona escribe. El `<label>` es el texto descriptivo ("Escribe tu nombre aquí"). Un error gigantesco de principiante es poner texto suelto al lado del input. Lo profesional es conectarlos usando los atributos `for` e `id`.

- **¿Por qué es tan importante conectarlos?** Si los vinculas correctamente, cuando el usuario haga clic torpemente en la palabra del label en su celular, el cursor saltará mágicamente adentro de la cajita del input. Mejora brutalmente la experiencia de usuario (UX).

El contenedor `<form>` necesita dos atributos vitales de configuración: `action` (la URL del servidor a dónde se envía la carta) y `method` (en qué medio de transporte enviamos la carta). Hay dos métodos principales de transporte:

Característica	Método <code>GET</code> (Público y Rápido)	Método <code>POST</code> (Privado y Seguro)
¿Cómo viajan los datos?	Viajan visibles y pegados en la URL de arriba del navegador (ej: <code>/buscar?zapatos=rojos</code>).	Los datos viajan ocultos en las entrañas (cuerpo) del mensaje hacia el servidor.
Nivel de Seguridad	Inseguro. Cualquiera que mire la pantalla puede ver la información en la barra de direcciones.	Seguro. Es el único método legal y válido para enviar contraseñas o tarjetas de crédito.
Historial y Favoritos	Al estar en la URL, se guardan en el historial. Puedes copiar el link y pasárselo a un amigo.	Al ser oculto, no se puede compartir ni guardar en favoritos.
Caso de Uso Real	Buscadores web (como Google Search) o filtros de productos en una tienda online.	Registros de usuarios, Inicios de sesión (Login) y envío de imágenes pesadas.

Tipos de Inputs más usados en el mercado

El atributo `type` cambia completamente el comportamiento visual y funcional del input:

- `type="text"`: Para textos genéricos cortos (nombres, marcas).
- `type="password"`: Automáticamente oculta lo que escribes con asteriscos o puntos por privacidad.
- `type="email"`: El navegador hace una validación gratuita para asegurar que el texto contenga un `@` antes de permitir el envío.
- `type="radio"`: Botones circulares de opción de exclusión mutua (ej: elegir tarjeta O efectivo). **Regla de oro:** Para que funcionen en grupo y solo puedas elegir uno, todos los radios del grupo deben tener el mismo atributo `name`.
- `type="submit"`: El botón mágico ("Enviar"). Cuando lo aprietas, busca su etiqueta padre `<form>`, empaqueta todo y lo despacha al servidor.

Ejemplo de un Formulario Estructurado Perfecto:

```
<!-- Enviamos los datos ocultos por POST al servidor imaginario /registro -->
<form action="/registro" method="POST">

  <!-- El atributo 'for' debe ser IDÉNTICO al 'id' del input para vincularlos -->
  <label for="nombre_usuario">Nombre Completo:</label>
  <input type="text" id="nombre_usuario" name="nombre_real"
placeholder="Ej: Juan Perez" required>

  <label for="clave">Crea una Contraseña:</label>
  <input type="password" id="clave" name="password_secreta" required>

  <label for="correo">Tu E-mail de contacto:</label>
  <input type="email" id="correo" name="email_contacto">
```

```
<!-- Agrupación Semántica de opciones (Radio Buttons) -->
<fieldset>
  <legend>Elige tu Plan de Suscripción</legend>

  <!-- Observa que ambos tienen name="plan" para que compitan entre
sí -->
  <input type="radio" name="plan" id="basico" value="plan_basico">
  <label for="basico">Plan Básico (Gratis)</label>

  <input type="radio" name="plan" id="pro" value="plan_pro">
  <label for="pro">Plan Profesional ($10)</label>
</fieldset>

<button type="submit">Enviar mis datos al Servidor</button>
</form>
```

⚠ IMPORTANTE

El rol fundamental del atributo `name`: Observa el código anterior. Cada input tiene obligatoriamente el atributo `name`. Si lo olvidas, el input se mostrará, pero el servidor recibirá el sobre vacío. El `name` es la "etiqueta interna" con la que el programador del Backend identificará y guardará el dato en la base de datos (ej: `request.body.email_contacto`)

Tema 3: Implementación de audio, video y figuras.

3.1. Multimedia, Elementos Embebidos y Control de Visibilidad

Ya tenemos la estructura semántica, ahora démosle vida a la web con medios visuales e interactivos. El manejo de imágenes, audios, videos y la capacidad de integrar recursos externos o controlar cómo se muestran en pantalla de forma nativa nos brinda las herramientas esenciales para maquetar interfaces ricas, dinámicas y profesionales.

Para insertar una imagen, usamos la etiqueta de auto-cierre `<img>`.

```

```

⚠ IMPORTANTE

El atributo `alt` (Texto Alternativo) es **estrictamente obligatorio** en el mundo profesional. Cumple dos funciones críticas: Si la foto se rompe o el internet es muy lento, se muestra ese texto. Y más importante aún, es lo que el lector de pantalla le leerá a una persona ciega para que pueda "imaginar" la foto. ¡Nunca lo dejes vacío!

En la web moderna no necesitamos plugins de terceros (como el antiguo y extinto Adobe Flash) para reproducir videos. HTML5 nos da etiquetas nativas. Ambas etiquetas (`<audio>` y `<video>`) comparten los mismos atributos clave de control:

Atributo	Función Visual / Técnica
src	La ruta absoluta o relativa hacia tu archivo de video (.mp4) o audio (.mp3).
controls	Muestra los botones clásicos del reproductor (Play, Pausa, Volumen). Sin esto, tu video existirá pero el usuario no podrá ponerle Play.
autoplay	Fuerza a que se reproduzca solo apenas carga la página (Cuidado: esto es muy invasivo para el usuario, úsalo con suma moderación).
loop	Hace que el video se reinicie automáticamente al terminar (bucle infinito).
muted	Obliga a que inicie sin sonido. (Nota técnica: Google Chrome y Safari modernos obligan a usar muted si intentas usar autoplay para no asustar al usuario con ruido repentino).

Ejemplo de Video de Fondo (Sin controles, mudo e infinito):

```
<video src="videos/lluvia-fondo.mp4" width="100%" autoplay muted loop>
  Lo sentimos, tu navegador es muy antiguo y no soporta el tag de video.
</video>
```

A la par de estos recursos nativos, los `<iframe>` son como "portales mágicos". Nos permiten incrustar una página web entera (o partes interactivas como Google Maps o videos de YouTube) dentro de un rectángulo de nuestra propia página. Técnicamente, estás cargando un sitio web ajeno dentro del tuyo.

Ejemplo de incrustación de un Mapa interactivo: (Este código se genera automáticamente en Google Maps usando la opción "Compartir -> Insertar un mapa").

```
<iframe src="https://www.google.com/maps/embed?pb=..." width="600"  
height="450" style="border:0;" allowfullscreen="" loading="lazy"></iframe>
```

Para complementar estas integraciones, a veces necesitaremos ocultar temporalmente una alerta, un menú lateral o una ventana emergente. Hay dos formas principales de esconder algo, y la diferencia estructural es gigantesca (Concepto Previo a CSS):

Regla <u>CSS</u>	¿Qué efecto visual tiene?	¿Qué pasa con el espacio en la página?
<code>display: none;</code>	El elemento desaparece totalmente de la vista.	Físicamente desaparece. Los demás elementos se acomodan como si nunca hubiera existido.
<code>visibility: hidden;</code>	El elemento se vuelve invisible (como el hombre invisible).	Mantiene el hueco. El elemento sigue ocupando su espacio físico exacto, dejando un gran agujero en blanco en la web.

El dominio de los elementos multimedia, los portales de inserción externa y los principios de visualización en pantalla te otorgan un control integral sobre los componentes interactivos de una web. Al asimilar estas herramientas de maquetación avanzada junto con las reglas semánticas previas, dispones de todos los fundamentos técnicos necesarios para diseñar interfaces modernas que respondan con precisión a las demandas estéticas y de accesibilidad actuales de la industria.

Documentación Oficial (Referencias)

- **CSS:** <https://developer.mozilla.org/es/docs/Web/CSS>
- **HTML:** <https://developer.mozilla.org/es/docs/Web/HTML>
- **JS:** <https://developer.mozilla.org/es/docs/Web/JavaScript>